

ITI · INTRODUCTION TO PYTHON TRACK

Python Lab 1

Foundations for AI Applications

Hands-on exercises to practice the Python fundamentals every AI engineer relies on daily: identifiers, variables, data types, operators, strings, and core data structures.

Exercises

01

Identifier & Variable Roll Call

Declare variables for a model's name (string), its accuracy (float), whether it's currently deployed (boolean), and its version number (int) — using valid, descriptive identifiers. Print each variable together with its type using `type()`.

- *Identifiers can only start with a letter or underscore, and can't be a reserved word.*
- *`type(x)` tells you exactly what Python thinks a variable is.*

AI connection: These are the same categories of variables you'll define when configuring any AI model or experiment.

02

Type Conversion Station

Given two raw strings: `raw_accuracy = "94.7"` and `raw_epochs = "10"`, convert them to the correct numeric types and print a single formatted sentence (using an f-string) that reports both values together.

- *`input()` and data read from files always arrive as strings — casting with `int()/float()` is essential.*
- *`f"{variable}"` lets you drop a variable straight into a sentence.*

AI connection: Every time you load a CSV or a config file, numeric-looking values often arrive as text and need this exact conversion before they can be used in a model.

03

Operators Workout

Given `x = 17` and `y = 5`, print the results of `+`, `-`, `*`, `/`, `%`, `**`, and `//`. Then, starting from `total = 0`, apply `+=`, `-=`, and `*=` one after another, printing the running result after each step.

- *`%` gives you the remainder; `//` gives you the whole-number result of division.*

- `x += 3` is shorthand for `x = x + 3`.

AI connection: Accumulating a running total with `+=` is exactly the pattern used to track loss across batches during model training.

04

String Toolbox

Given `model_name = " GPT-4 "`, clean it up: strip the extra whitespace, make it fully uppercase, and replace the hyphen with an underscore. Then print the length of the cleaned string and use slicing to print just the first three characters.

- `.strip()`, `.upper()`, and `.replace(old, new)` all return a new string — they don't change the original.
- Slicing a string works the same as slicing a list: `s[start:end]`.

AI connection: This exact sequence — strip, normalize case, replace characters — is the first step of almost every text-preprocessing pipeline before feeding text into an NLP model.

05

List Lab

Create a list of five prediction scores: `scores = [0.71, 0.85, 0.63, 0.90, 0.78]`. Append one more score, replace the first score with a new value using its index, and print a slice containing just the last three scores. Finally, print the list's length.

- `.append(value)` adds to the end; `scores[0] = new_value` edits in place.
- Negative-index slicing (`scores[-3:]`) is handy for grabbing 'the last N' items.

AI connection: A batch of model predictions or a mini-batch of training samples is almost always stored and manipulated as a Python list.

06

Tuple Vault

Create a tuple describing an image's shape: `image_shape = (224, 224, 3)`. Print each element by indexing (height, width, channels separately) and, in a comment, explain what error Python would raise if you tried to change one of its values.

- Tuples use the same indexing as lists — `image_shape[0]`, `image_shape[1]`, etc.
- You don't need to actually trigger the error — just describe it in a comment.

AI connection: Fixed structural values like image dimensions or a model's input shape should never change by accident — that's exactly what a tuple protects against.

07

Dictionary Depot

Build a dictionary representing a model's hyperparameters: `config = {'learning_rate': 0.001, 'batch_size': 32}`. Add a new key 'epochs' with the value 10, update 'batch_size' to 64, then print the whole dictionary, just its keys, and just its values.

- *Add or update a key with `config['new_key'] = value`.*
- *`.keys()` and `.values()` give you those two views without needing a loop.*

AI connection: Hyperparameter configurations in virtually every ML framework — *scikit-learn*, *PyTorch*, *TensorFlow* — are just Python dictionaries under the hood.

08

Mini Challenge — Model Report Card

Combine everything so far into one nested structure:

- *`report = {'name': 'QuantumStride', 'accuracy': 0.912, 'metrics': {'precision': 0.89, 'recall': 0.87}, 'sample_predictions': [0.81, 0.42, 0.95]}`*
- *Then print the model's name, its nested precision value, and the second sample prediction — using only indexing and key lookups (no loops needed).*
- *Chain lookups directly: `report['metrics']['precision']`.*
- *List indexing works inside a dictionary value just like anywhere else: `report['sample_predictions'][1]`.*

AI connection: This is essentially what a real experiment-tracking record (like a *Weights & Biases* or *MLflow log*) looks like under the hood.

Quick Practice — Warm-up Problems

If you finish early, try these short problems to reinforce today's syntax:

- Write a Python program to add two numbers and print the result.
- Swap the values of two variables without using a third temporary variable (hint: `a, b = b, a`).
- Reverse a string using slicing — `s[::-1]` — no loop needed.
- Convert a list of numbers into a tuple, and then back into a list, using `tuple()` and `list()`.